

Conditionals and Loops

Conditionals in Bash

Conditionals are constructs, which allow you to test for a specific condition

In Bash, the 'if' statement is used to execute a block of code based on a condition. The basic syntax for an 'if' statement is:

```
if [ condition ]; then
    # code to be executed if the condition is true
fi
```

Here's an example:

```
age=18
if [ $age -ge 18 ]; then
    echo "You are an adult."
fi
```

In this example, the condition `[ $age -ge 18 ]` checks if the value of the `age` variable is greater than or equal to 18. If the condition is true, the code inside the `then` block will be executed, and the message "You are an adult." will be printed.

You can also use the 'else' statement to execute a different block of code if the condition is false:

```
age=16
if [ $age -ge 18 ]; then
    echo "You are an adult."
else
    echo "You are a minor."
fi
```

In this example, since the `age` variable is 16, which is less than 18, the condition `[ $age -ge 18 ]` is false, and the code inside the `else` block will be executed, printing the message "You are a minor."

You can also use multiple 'elif' (else if) statements to check for additional conditions:

```
age=25
if [ $age -lt 18 ]; then
    echo "You are a minor."
elif [ $age -ge 18 ] && [ $age -lt 65 ]; then
    echo "You are an adult."
else
    echo "You are a senior."
fi
```

In this example, the first condition `[ $age -lt 18 ]` checks if the `age` variable is less than 18. If it is, the code inside the first `if` block will be executed, and the message "You are a minor." will be printed.

If the first condition is false, the second condition

`[ $age -ge 18 ] && [ $age -lt 65 ]` is checked. This condition checks if the `age` variable is greater than or equal to 18 and less than 65. If this condition is true, the code inside the `elif` block will be executed, and the message "You are an adult." will be printed.

If both the first and second conditions are false, the `else` block will be executed, and the message "You are a senior." will be printed.

Here are a few more examples to help you understand the use of 'if' and 'else' statements in Bash:

```
# Example 1: Checking if a file exists
file="example.txt"
if [ -e "$file" ]; then
    echo "The file $file exists."
else
    echo "The file $file does not exist."
fi

# Example 2: Checking if a directory is empty
directory="/path/to/directory"
if [ -z "$(ls -A "$directory")" ]; then
    echo "The directory $directory is empty."
else
    echo "The directory $directory is not empty."
fi

# Example 3: Checking if a variable is empty
name=""
if [ -z "$name" ]; then
    echo "The name variable is empty."
else
    echo "The name variable is not empty."
```

```
fi
```

Loops in Bash

A loop is a programming construct that allows you to execute a block of code multiple times. This is useful when you want to perform the same action repeatedly without having to write the same code over and over again.

Types of Loops in Bash

Bash supports several types of loops, but we will focus on the three most common ones:

1. **For Loop**
2. **While Loop**
3. **Until Loop**

1. For Loop

The `for` loop is used to iterate over a list of items. Here's the basic syntax:

```
for variable in list
do
    # commands to be executed
done
```

Example 1: Simple For Loop

Let's create a simple `for` loop that prints numbers from 1 to 5.

```
#!/bin/bash

for i in 1 2 3 4 5
do
    echo "Number: $i"
done
```

Explanation:

- `for i in 1 2 3 4 5`: This line defines a loop that will iterate over the numbers 1 to 5.

- `echo "Number: $i"` : This command prints the current value of `i`.

Example 2: For Loop with a Range

You can also use a range of numbers with the `for` loop.

```
#!/bin/bash

for i in {1..5}
do
    echo "Number: $i"
done
```

Explanation:

- `{1..5}` generates a sequence of numbers from 1 to 5.

2. While Loop

The `while` loop continues to execute as long as a specified condition is true. Here's the basic syntax:

```
while condition
do
    # commands to be executed
done
```

Example 3: Simple While Loop

Let's create a `while` loop that counts from 1 to 5.

```
#!/bin/bash

i=1
while [ $i -le 5 ]
do
    echo "Number: $i"
    ((i++)) # Increment i by 1
done
```

Explanation:

- `i=1` : Initializes the variable `i` to 1.

- `while [ $i -le 5 ]` : The loop continues as long as `i` is less than or equal to 5.
- `((i++))` : Increments `i` by 1.

3. Until Loop

The `until` loop is similar to the `while` loop, but it continues until a specified condition becomes true. Here's the basic syntax:

```
until condition
do
    # commands to be executed
done
```

Example 4: Simple Until Loop

Let's create an `until` loop that counts from 1 to 5.

```
#!/bin/bash

i=1
until [ $i -gt 5 ]
do
    echo "Number: $i"
    ((i++)) # Increment i by 1
done
```

Explanation:

- `until [ $i -gt 5 ]` : The loop continues until `i` is greater than 5.

Nested Loops

You can also nest loops within each other. Here's an example of a nested `for` loop that creates a multiplication table.

```
#!/bin/bash

for i in {1..5}
do
    for j in {1..5}
    do
        echo "$i * $j = $((i * j))"
```

```
done
done
```

Explanation:

- The outer loop iterates over `i`, and for each value of `i`, the inner loop iterates over `j`, creating a multiplication table.

Exercises

1. Write a script that counts the number of arguments, and outputs

```
You provided an even number of arguments
```

 OR

```
You provided an odd number of arguments
```

, depending on the situation. If the user provided zero arguments, then the script should say

```
Please provide some arguments
```

 and give the user one more chance.

2. Write a script that tests if:
 - a. The user provided one argument exactly, else it should exit with an error message
 - b. If the provided argument is a file or a directory
 - c. Depending on the result for (b), it should provide an appropriate error message and exit